

Angular, SPA & Webpack Overview

Lifecycle Hooks • Single Page Applications •
Webpack SPA • Angular App Examples

What is Angular?

- Angular is a TypeScript-based frontend framework by Google
 - Used to build Single Page Applications (SPAs)
 - Provides components, services, routing, dependency injection
 - Uses HTML + TypeScript + CSS

Angular Component Lifecycle

- Lifecycle = stages of a component from creation to destruction
 - Angular provides lifecycle hooks to run logic at each stage
 - Helps manage initialization, updates, and cleanup

Lifecycle Hooks Order

- `constructor()`
 - `ngOnChanges()`
 - `ngOnInit()`
 - `ngDoCheck()`
 - `ngAfterContentInit()`
 - `ngAfterViewInit()`
 - `ngOnDestroy()`

Most Used Lifecycle Hooks

- constructor – Dependency Injection only
 - ngOnInit – API calls, initial data
 - ngOnChanges – React to @Input changes
 - ngAfterViewInit – DOM / ViewChild access
 - ngOnDestroy – Cleanup, unsubscribe

Lifecycle Example

- Parent passes data using @Input
 - Child receives data and logs lifecycle hooks
 - Destroy/Create child using *ngIf
 - Observe lifecycle logs in browser console

What is a Single Page Application (SPA)?

- Application loads a single HTML page
 - Content updates dynamically without page reload
 - Routing handled on client-side using JavaScript
 - Examples: Angular, React, Vue apps

SPA vs Traditional Web App

- Traditional: Server reloads page on every request
 - SPA: Only data changes, page stays the same
 - Faster navigation and better user experience

Webpack SPA

- Webpack bundles JavaScript/TypeScript files
 - Uses custom router with History API or Hash routing
 - Loads views dynamically without reload
 - Used before frameworks or for learning internals

Webpack SPA Example

- home.ts, about.ts, contact.ts – views
 - routes.ts – path to view mapping
 - router.ts – navigation logic
 - index.html – single HTML page

Routing in Webpack SPA

- Uses hash (#/about) or History API (/about)
 - Live Server supports hash routing only
 - Webpack Dev Server supports historyApiFallback

My Angular App

- Created using Angular CLI (ng new)
 - Uses standalone components
 - Component-based architecture
 - Built-in routing and lifecycle support

Angular App Example

- Parent component passes data using @Input
 - Child emits events using @Output or output()
 - Lifecycle hooks logged in console
 - ngIf used to destroy/create components

Angular vs Webpack SPA

- Angular provides built-in routing and lifecycle
 - Webpack SPA requires manual routing logic
 - Angular is scalable and enterprise-ready
 - Webpack SPA is lightweight and educational

Key Points

- Angular lifecycle manages component stages
 - SPAs update content without page reload
 - Webpack SPA demonstrates client-side routing
 - `ngOnInit` and `ngOnDestroy` are most important hooks